BONA IT

IT Support Service Desk Portal

Project Implementation Report

Project ID: scope-002

Submitted to: Nathan Anvis

Organisation: Global Software Services — Student Projects Programme

Date: July 2026

Repository: github.com/Thuso4747/bona-it-service-desk

Deployment: https://bona-it-service-desk-system.vercel.app/

Project Team

Name	Role
George Fourie	Database Architect / System Integration
Thuso Rampheng	Email Specialist / System Integration / UI Lead
Thamsanqa Yawa	API Developer / Testing
Collins	Client UI Developer
Koketso	Agent UI Developer
Monde	Security & Authenticity Lead

1. Project Overview

Bona IT is a branded IT support service desk portal developed as part of the Global Software Services Student Projects Programme. The system enables clients to log IT support requests via a web form or email, track ticket status using a unique token reference, and receive automated confirmation emails. Support agents manage and resolve tickets through a secure internal dashboard.

The project was developed over four phases — Planning & Setup, Core Implementation, Testing & Polish, and Deployment & Documentation — with a team of eight developers each responsible for a specific domain of the system.

2. Technology Stack

Technology	Purpose
React + Vite	Frontend framework for client simulator and agent dashboard
TypeScript	Type-safe development across frontend and backend
Tailwind CSS	Utility-first styling for responsive UI components
Prisma ORM	Database schema definition, migrations, and type-safe queries
PostgreSQL (Neon)	Cloud-hosted relational database via Neon serverless platform
Nodemailer + Mailtrap	Email sending and inbox simulation for ticket ingestion
Vercel	Deployment platform for the production application
GitHub	Version control and team collaboration

3. Database Architecture

The database was designed and implemented by George Fourie (Database Architect) using Prisma ORM with a PostgreSQL database hosted on Neon. The schema was designed from first principles to support all system requirements including role-based access, ticket lifecycle management, and token-based tracking.

3.1 Design Decisions

A key architectural decision was to use a single **User** table for all user types (clients, agents, and admins) rather than separate tables. This was achieved using a **Role** enum, which simplifies queries and avoids data duplication while still allowing role-based access control.

Similarly, a single **Ticket** table was used for all ticket states rather than separate tables per status. A **TicketState** enum tracks the ticket lifecycle (CREATED → PROCESSING → COMPLETED), allowing full history to be preserved without moving records between tables.

3.2 Schema Overview

User Table

Field	Type	Description
id	Int (PK)	Auto-incrementing primary key
userRef	String (Unique)	Human-readable reference e.g. USR-79EBE9D2
name	String	User's full name
email	String (Unique)	Login email address
password	String	User password
role	Role (Enum)	CLIENT, AGENT, or ADMIN

Ticket Table

Field	Type	Description
id	Int (PK)	Auto-incrementing primary key
ticketRef	String (Unique)	Human-readable reference e.g. TKT-A2AFF7CC
trackingToken	String (Unique)	UUID token for client status tracking without login
title	String	Short summary of the issue
description	String	Full details of the support request
reportType	ReportType (Enum)	SUGGESTION, COMPLAINT, FEEDBACK, or OTHER
status	TicketState (Enum)	CREATED, PROCESSING, or COMPLETED
creationDate	DateTime	Auto-set timestamp on ticket creation
updatedDate	DateTime	Auto-updated on every change
slaDeadline	DateTime (Optional)	Response deadline for SLA tracking
submittedBy	Int (FK → User)	Links to the client who submitted the ticket
assignedTo	Int (Optional FK → User)	Links to the agent handling the ticket
resolvedBy	Int (Optional FK → User)	Links to the agent who resolved the ticket

4. API Design

The backend API was built by Thamsanqa (API Developer 1) and Miles (API Developer 2) as an Express server (server.ts), with matching serverless functions under /api for the Vercel deployment. The API provides endpoints for authentication, ticket creation, retrieval, status updates, and user management.

Key Endpoints:

POST /api/auth/login — Authenticates a user and returns their profile and role

POST /api/auth/signup — Registers a new client account

POST /api/tickets — Creates a new ticket from the client submission form

POST /api/webhooks/tickets — Converts an incoming support email into a ticket automatically (see Section 5)

GET /api/tickets — Retrieves all tickets, joined with submitter details, for the agent dashboard

GET /api/tickets/status/:token — Returns ticket status by tracking token or ticket reference (no login required)

PATCH /api/tickets/updates — Agent-only endpoint that updates a ticket's status and sends a notification email to the client

PATCH /api/tickets/:id — Updates a ticket's title and description

DELETE /api/tickets/:id — Removes a ticket

GET /api/users, PATCH /api/users/:id, DELETE /api/users/:id — User management endpoints used by the agent dashboard

The token tracking endpoint is particularly notable as it allows clients to check their ticket status without requiring authentication, using the unique **trackingToken** generated at ticket creation. The endpoint also supports automatic normalisation of 8-character hex codes to TKT- format.

4.1 Example Request & Response

Representative request and response payloads for the two most-used endpoints are shown below, reflecting the actual Prisma field names used in the schema (Section 3.2).

Ticket Creation — POST /api/tickets

```
Request:
{
  "name": "Sebastian",
  "email": "client@example.com",
  "description": "Office printer on the 3rd floor is not
responding to print jobs.",
  "reportType": "COMPLAINT"
}

Response:
{
  "success": true,
  "token": "b3f1c9e2-4a2d-4e91-9c31-7a6e2f0d1a44",
  "ticketRef": "TKT-A2AFF7CC",
  "status": "CREATED",
  "emailSent": true
}
```

Ticket Status Lookup — GET /api/tickets/status/:token

```
Example: /api/tickets/status/TKT-A2AFF7CC

Response:
{
  "success": true,
  "token": "b3f1c9e2-4a2d-4e91-9c31-7a6e2f0d1a44",
  "ticketRef": "TKT-A2AFF7CC",
  "title": "Printer not working",
  "status": "PROCESSING",
  "submittedBy": { "name": "Sebastian", "email":
"client@example.com" }
}
```

4.2 Request Flow

Every API call follows the same path from client to database and back:

Client Request → Express API Route → node-postgres (pg) Query → Neon PostgreSQL → JSON Response

The Express API route receives and validates the incoming request, then executes a parameterised SQL query against the Neon-hosted PostgreSQL database using the node-postgres (pg) driver, and returns the result as a JSON response. The Prisma schema (Section 3.2) defines the table structure and drives migrations, while the live API queries the database directly rather than through the generated Prisma Client. This replaced an earlier in-memory prototype used during initial API development, giving the system persistent storage from Phase 2 onward.

5. Email Integration

Email integration was handled by Thuso Rampheng (Email Specialist). The system supports two email flows:

Email-to-Ticket Ingestion:

The system monitors an inbox and automatically converts incoming emails into support tickets. Each ingested email generates a unique ticket reference and tracking token, and triggers an auto-reply confirmation to the sender.

Status Update Notifications:

When an agent updates a ticket's status, the system sends an automated email notification to the client with the new status and ticket reference. Mailtrap was used as a sandbox email provider during development to simulate sending and receiving emails safely.

6. Deployment

The application was deployed to Vercel by Thuso Rampheng. The production database is hosted on Neon (serverless PostgreSQL). Environment variables including the **DATABASE_URL** connection string are configured in Vercel's dashboard and are never committed to the repository.

Live URL: https://bona-it-service-desk-system.vercel.app/

Agent Login: admin@portal.com / admin123

7. Challenges & Solutions

Challenge	Solution
Working with unfamiliar team members in a new environment	Regular communication via WhatsApp group and structured task assignments per phase helped the team coordinate effectively despite having no prior working relationship
New coding languages and tools (Next.js, TypeScript, Prisma, Git)	Each team member focused on their assigned domain, learning the specific tools relevant to their role rather than trying to learn everything at once
Database integration across independently built components	A centralised Prisma schema was established early as the source of truth, with all team members aligning their code to the shared field names and types
Email ingestion bypassing the Prisma schema	The email pipeline was initially built against a separate database structure. This was resolved by migrating the shared schema to Neon and connecting the Vercel deployment to the unified database
Git merge conflicts on shared files	Package.json and README conflicts were resolved manually by identifying and keeping the correct version of each conflicting section
Prisma script execution policy blocked on Windows	Resolved by updating the PowerShell execution policy using Set-ExecutionPolicy RemoteSigned
Some team members were unable to contribute due to technical issues (laptop failures) and communication gaps	Thuso Rampheng took on additional responsibilities, compiling and integrating the final system, handling deployment, and producing the demo video to ensure the project met submission requirements

8. Deliverables Checklist

Deliverable	Status
Client ticket submission form	✓ Complete
Email-to-ticket ingestion with auto-reply	✓ Complete
Agent dashboard with ticket list, assignment, and status update	✓ Complete
Token-based client ticket status tracking page	✓ Complete
Email notifications on ticket events	✓ Complete
Basic SLA indicator (response deadline)	✓ Complete
Deployed application with demo credentials	✓ Complete

9. Conclusion

The Bona IT Service Desk Portal was successfully designed, developed, and deployed within the project timeline. The system meets all specified deliverable requirements and demonstrates a functional end-to-end IT support workflow — from ticket submission through email or web form, to agent management and client status tracking.

The project presented significant learning opportunities for all team members, particularly in areas of collaborative development using Git, working with new frameworks and cloud platforms, and integrating independently built components into a unified system. Despite the challenges of working with an unfamiliar team in a new technical environment, the team successfully delivered a production-ready application.

The database architecture established in Phase 1 proved to be a solid foundation that supported the entire system — the Prisma schema defined by the Database Architect became the single source of truth that all other components aligned to, demonstrating the importance of early, well-structured data modelling in team software projects.